

Question 01:

- Consider two different machines, with two different instruction sets, both of which have a clock rate of 200 MHz. The following measurements are recorded on the two machines running a given set of benchmark programs:

Instruction Type	Instruction Count (millions)	Cycles per Instruction
Machine A		
Arithmetic and logic	8	1
Load and store	4	3
Branch	2	4
Others	4	3
Machine B		
Arithmetic and logic	10	1
Load and store	8	2
Branch	2	4
Others	4	3

Find the CPI (Average cycle per instruction), MIPS rate and CPU execution time for the two machines.

$$CPI_A = \frac{\sum CPI_i \times I_i}{I_c} = \frac{(8 \times 1 + 4 \times 3 + 2 \times 4 + 4 \times 3) \times 10^6}{(8 + 4 + 2 + 4) \times 10^6} = 2.22$$

$$MIPS_A = \frac{f}{CPI_A \times 10^6} = \frac{200 \times 10^6}{2.22 \times 10^6} = 90$$

$$CPU_A = \frac{I_c \times CPI_A}{f} = \frac{18 \times 10^6 \times 2.2}{200 \times 10^6} = 0.2 \text{ s}$$

$$CPI_B = \frac{\sum CPI_i \times I_i}{I_c} = \frac{(10 \times 1 + 8 \times 2 + 2 \times 4 + 4 \times 3) \times 10^6}{(10 + 8 + 2 + 4) \times 10^6} = 1.92$$

$$MIPS_B = \frac{f}{CPI_B \times 10^6} = \frac{200 \times 10^6}{1.92 \times 10^6} = 104$$

$$CPU_B = \frac{I_c \times CPI_B}{f} = \frac{24 \times 10^6 \times 1.92}{200 \times 10^6} = 0.23 \text{ s}$$

- Two different compilers are being tested on the same program for a 4 GHz machine with three different classes of instructions: Class A, Class B, and Class C, which require 1, 2, and 3 cycles, respectively. The instruction count produced by the first compiler is 5 billion Class A instructions, 1 billion Class B instructions, and 1 billion Class C instructions. The second compiler produces 10 billion Class A instructions, 1 billion Class B instructions, and 1 billion Class C instructions. Which compiler produces a higher MIPS? Which compiler produces a better execution time?

First, we find the CPU cycles for both compilers

- CPU cycles (compiler 1) = $(5 \times 1 + 1 \times 2 + 1 \times 3) \times 10^9 = 10 \times 10^9$
- CPU cycles (compiler 2) = $(10 \times 1 + 1 \times 2 + 1 \times 3) \times 10^9 = 15 \times 10^9$

Next, we find the execution time for both compilers:

Execution time = number of cycles / cpu frequency

- Execution time (compiler 1) = $10 \times 10^9 \text{ cycles} / 4 \times 10^9 \text{ Hz} = 2.5 \text{ sec}$
- Execution time (compiler 2) = $15 \times 10^9 \text{ cycles} / 4 \times 10^9 \text{ Hz} = 3.75 \text{ sec}$

Compiler 1 generates faster program (less execution time)

Now, we compute MIPS rate for both compilers

- MIPS = Instruction Count / (Execution Time $\times 10^6$).**
- MIPS (compiler 1) = $(5+1+1) \times 10^9 / (2.5 \times 10^6) = 2800$
- MIPS (compiler 2) = $(10+1+1) \times 10^9 / (3.75 \times 10^6) = 3200$

So, code from compiler 2 has a higher MIPS rating !!!

- Consider the execution of a program which results in the execution of 2 million instructions on a 400-MHz processor. The program consists of four major types of instructions. The instruction mix and the CPI for each instruction type are given below based on the result of a program trace experiment:

Instruction Type	CPI	Instruction Mix
Arithmetic and logic	1	60%
Load/store with cache hit	2	18%
Branch	4	12%
Memory reference with cache miss	8	10%

- Determine the average CPI.
- Determine the corresponding MIPS rate.

Solution:

The average CPI when the program is executed on a uniprocessor with the above trace results is $CPI = 0.6 + (2 \times 0.18) + (4 \times 0.12) + (8 \times 0.1) = 2.24$. The corresponding MIPS rate is $(400 \times 10^6) / (2.24 \times 10^9) \approx 178$.

- Consider the example in the previous exercise for the calculation of average CPI and MIPS rate, which yielded the result of CPI 2.24 and MIPS rate 178. Now assume that the program can be executed in eight parallel tasks or threads with roughly equal number of instructions executed in each task. Execution is on an 8-core system with each core (processor) having the same performance as the single processor originally used. Coordination and synchronization between the parts adds an extra 25,000 instruction executions to each task. Assume the same instruction mix as in the example for each task, but increase the CPI for memory reference with cache miss to 12 cycles due to contention for memory.

- Determine the average CPI.

Since we have the same instruction mix, that means the additional instructions for each task could be allocated appropriately between the instruction types. Therefore, the following table be gotten:

Instruction Type	CPI	Instruction Mix
Arithmetic and logic	1	60%
Load/store with cache hit	2	18%
Branch	4	12%
Memory reference with cache miss	8 → 12	10%

The average CPI = $(1 \times 0.6) + (2 \times 0.18) + (4 \times 0.12) + (12 \times 0.1) = 2.64$.

Therefore, the CPI has been increased since the time for memory access is also increased.

- Determine the corresponding MIPS rate.

MIPS = $400 / 2.64 = 152$. There is a corresponding drop in the MIPS rate.

- c. Calculate the speedup factor.

The speedup factor equals to the ratio of the execution times.

The execution times is calculated as the following: $T = \frac{I_e}{MIPS \times 10^6}$

For the one processor: $T_1 = (2 \times 10^6) / (178 \times 10^6) = 11\text{ms}$

For the 8 processors, each processor executes 1/8 of the 2 million instructions plus the 25,000 overhead instructions. There, the execution time T for each of the 8 processors is:

$$T_8 = \frac{\frac{2 \times 10^6}{8} + 0.025 \times 10^6}{152 \times 10^6} = 1.8 \text{ ms}$$

Therefore, we have

$$\text{speedup} = \frac{\text{time to execute program on a single processor}}{\text{time to execute program on } N \text{ parallel processors}} = \frac{11}{1.8} = 6.11$$

- d. Compare the actual speedup factor with the theoretical speedup factor determined by Amdahl's law

Solution:

d. In fact, there are two inefficiencies in the parallel system. The first one is that there are more additional instructions which is added to coordinate between threads. The second one is that there is contention for memory access. Thus, none of the code is inherently serial, and all of it is parallelizable but with scheduling overhead. It could be said that the memory access conflict means some extent memory reference instructions are not parallelizable. By depending on the information given, it is not obvious how to quantify this effect in Amdahl's equation. Therefore, if it is supposed that the fraction of code which is parallelizable, is $f = 1$, then Amdahl's law decreases to $\text{Speedup} = N = 8$. Therefore, the actual speedup is only about 75% of the theoretical speedup.

5. A processor accesses main memory with an average access time of T_2 . A smaller cache memory is interposed between the processor and main memory. The cache has a significantly faster access time of $T_1 < T_2$. The cache holds, at any time, copies of some main memory words and is designed so that the words more likely to be accessed in the near future are in the cache. Assume that the probability that the next word accessed by the processor is in the cache is H , known as the hit ratio.
 - a. For any single memory access, what is the theoretical speedup of accessing the word in the cache rather than in main memory?
 - b. Let T be the average access time. Express T as a function of T_1 , T_2 , and H . What is the overall speedup as a function of H ?
 - c. In practice, a system may be designed so that the processor must first access the cache to determine if the word is in the cache and, if it is not, then access main memory, so that on a miss (opposite of a hit), memory access time is $T_1 + T_2$. Express

T as a function of T₁, T₂, and H. Now calculate the speedup and compare to the result produced in part (b).

a. $\text{Speedup} = (\text{time to access in main memory}) / (\text{time to access in cache})$
 $= T_2 / T_1$

b. The average access time could be calculated as the following:

$$T = H \times T_1 + (1 - H) \times T_2$$

By applying the following equation:

$$\begin{aligned} \text{Speedup} &= (\text{Execution time before enhancement}) / (\text{Execution time after enhancement}) \\ &= T_2 / T = T_2 / (H \times T_1 + (1 - H)T_2) \\ &= 1 / ((1 - H) + H \times (T_1 / T_2)) \end{aligned}$$

c. $T = H \times T_1 + (1 - H) \times (T_1 + T_2) = T_1 + (1 - H) \times T_2$

This is Equation in Chapter 4. Now,

$$\begin{aligned} \text{Speedup} &= (\text{Execution time before enhancement}) / (\text{Execution time after enhancement}) \\ &= T_2 / T = T_2 / (T_1 + (1 - H)T_2) = 1 / ((1 - H) + T_1 / T_2) \end{aligned}$$

Therefore, since the denominator is larger, the speedup will be less.

Question 02:

- The following memory units are specified by the number of words times the number of bits per word. How many address lines are needed in each case? How many input-output data lines are needed in each case?

Note:

- ✓ The number of address lines is determined by the number of addressable memory locations, which is calculated as the log base 2 of the total number of words.
- ✓ The number of data lines is simply the number of bits per word.
- ✓ To find the number of bytes stored in each memory, multiply the number of words by the number of bits per word, and then divide by 8 to convert bits to bytes.

a. 8 K x 16

8K x 16, there are 8,192 words, each containing 16 bits. Thus, there are 13 address lines ($2^{13} = 8,192$) and 16 data lines. The total storage capacity is $(8,192 \times 16) / 8 = 16,384$ bytes.

b. 4 G x 8

$$4G = 4 \times 2^{30} = 2^2 \times 2^{30} = 2^{32}$$

We need 32 address lines and 8 data lines.

c. 16 M x 32

$$16M = 16 \times 2^{20} = 2^4 \times 2^{20} = 2^{24}$$

We need 24 address lines and 32 data lines.

d. 256 K x 64

$$256K = 256 \times 2^{10} = 2^8 \times 2^{10} = 2^{18}$$

We need 18 address lines and 64 data lines.

2. Consider two microprocessors having 16- and 32-bit-wide external data buses, respectively. The two processors are identical otherwise and their bus cycles take just as long.

- a. Suppose all instructions and operands are four bytes long. By what factor do the maximum data transfer rates differ?

During a single bus cycle, the 16-bit microprocessor transfers two bytes while the 32-bit microprocessor transfers four bytes. The 32-bit microprocessor has twice the data transfer rate.

- b. Repeat assuming that 40% of the operands and instructions are two bytes long.

Suppose we do 100 transfers of operands and instructions, of which 40 are two bytes long and 60 are four bytes long. The 16-bit microprocessor takes $40 + (2 \times 60) = 160$ bus cycles for the transfer. The 32-bit microprocessor requires $40 + 60 = 100$ bus cycles. Thus, the data transfer rates differ by a factor of 1.6.

3. Consider a hypothetical 32-bit microprocessor having 32-bit instructions composed of two fields: the first byte contains the opcode and the remainder the immediate operand or an operand address.

- a. What is the maximum directly addressable memory capacity (in bytes)?

Opcode (1 byte)	Address (3 bytes)
-----------------	-------------------

3 bytes = 24 bits

$2^{24} = 2^{16} \times 2^8 = 4 \text{ Mbytes}$

- b. Discuss the impact on the system speed if the microprocessor bus has
- a 32-bit local address bus and a 16-bit local data bus, or
If the local address bus is 32 bits, the whole address can be transferred at once and decoded in memory.
However, since the data bus is only 16 bits, it will require 2 cycles to fetch a 32-bit instruction or operand.
 - a 16-bit local address bus and a 16-bit local data bus.
The 16 bits of the address placed on the address bus can't access the whole referenced word of memory. Thus a more complex memory interface control is needed to latch the first part of the address and then the second part.
In addition to the two-step address operation, the microprocessor will need 2 cycles to fetch the 32-bit instruction/operand since the data bus is only 16 bits.
- c. How many bits are needed for the program counter and the instruction register?
The program counter must be at least 24 bits.
The instruction register has to be 32-bits long, typically.

mory
instructions are executed. As
a result, one more output pin
will be required for I/O
signals.

If an input and an output
instruction can specify an
8-bit I/O port number, how
many 8-bit

I/O ports can the
microprocessor support?

How many 16-bit I/O ports?

Answer with Explanation:

The CPU can support $2^8 =$
256 8-bit input ports and 2^8
 $= 256$ 8-bit output ports with
an

8-bit I/O port number. The
microprocessor can support

$2^8 = 256$ 16-bit input ports
and 2

$^8 = 256$ 16-bit output ports
with an 8-bit I/O port number.

Because the number of I/O
ports depends on the
number of bits used to
encode the I/O port
number, changing the size of
the I/O port will not change

the number of I/O ports
(equals to 8
bits in both cases)
in memory
instructions are executed. As
a result, one more output pin
will be required for I/O
signals.
If an input and an output
instruction can specify an

8-bit I/O port number, how
many 8-bit
I/O ports can the
microprocessor support?
How many 16-bit I/O ports?
Answer with Explanation:
The CPU can support $2^8 =$
256 8-bit input ports and 2^8
 $= 256$ 8-bit output ports with
an

8-bit I/O port number. The microprocessor can support $2^8 = 256$ 16-bit input ports and $2^8 = 256$ 16-bit output ports with an 8-bit I/O port number. Because the number of I/O ports depends on the number of bits used to encode the I/O port

number, changing the size of the I/O port will not change the number of I/O ports (equals to 8 bits in both cases)

4. Consider a 32-bit microprocessor, with a 16-bit external data bus, driven by an 8-MHz input clock. Assume that this microprocessor has a bus cycle whose minimum duration equals four input clock cycles.

- a. What is the maximum data transfer rate across the bus that this microprocessor can sustain, in bytes/s?

Min bus cycle duration = four input clock cycles = $4 \times \text{input clock cycles duration}$

$$= 4 \times \frac{1}{\text{input clock frequency}} = 4 \times \frac{1}{8 \text{ MHz}} = 0.5 \times 10^{-6} \text{ sec.}$$

$$\text{maximum data transfer rate across the bus} = \frac{\# \text{ bytes of bus}}{\text{min bus cycle duration}}$$

$$= \frac{2 \text{ bytes}}{0.5 \times 10^{-6} \text{ sec}} = 4 \times 10^6 \text{ bytes/sec}$$

- b. To increase its performance, would it be better to make its external data bus 32 bits or to double the external clock frequency supplied to the microprocessor? State any other assumptions you make, and explain. Hint: Determine the number of bytes that can be transferred per bus cycle.

Doubling the frequency may require a new chip manufacturing technology (assuming each instruction will have the same number of clock cycles).

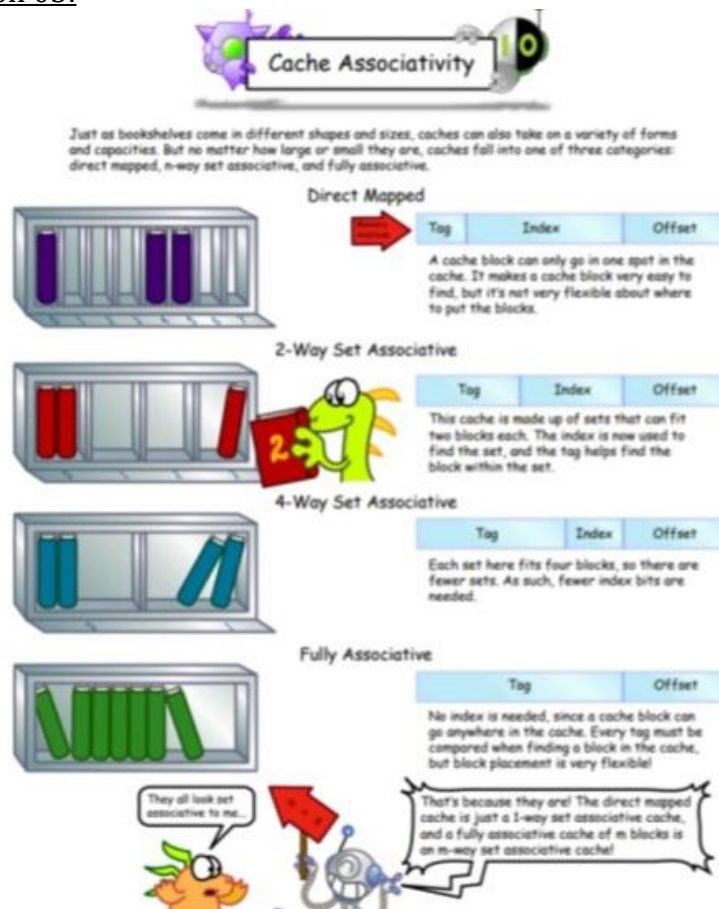
**** but we need to double the speed of the memory to prevent slowing the new microprocessor down.**

Doubling the external data bus means wider on-chip data bus drivers/latches and modifications to the bus control logic.

**** In this case, the “word length” of the memory will have to double to be able to send/receive 32-bit words.**

It would be better to make the external bus 32 bits. This would double the bit rate across the external bus by the least cost.

Question 03:



For purposes of cache access, each main memory address can be viewed as consisting of three fields.

- ✓ The least significant w bits (offset) identify a unique word or byte within a block of main memory
- ✓ The remaining s bits specify one of the 2^s blocks of main memory.

- o The cache logic interprets these s bits as a tag of $s - r$ bits (most significant portion) and a line field of r bits (index). This latter field identifies one of the $m = 2^r$ lines of the cache.
1. Given the following specifications for an external cache memory: four-way set-associative, line size of two 16-bit words; able to accommodate a total of 4K 32-bit words; used with 24-bit addresses. Design the cache structure.

Solution:

The idea is that we have the size of the address which is 24 bits, we need to know the size of the tag which is equal to (24 bits - the number of bits needed to identify a word in a line - number of bits needed to identify a set in the cache).

Since we have a four-way set thus this gives: 4 lines/set in the cache (every set is made up of 4 lines);

Line size is $2 \times 16 \text{ bits} = 32 \text{ bit} = 4 \text{ byte}$

$\Rightarrow$ 2 bits are needed to identify a byte in a line (because $4 = 2^2$)

The cache is made up of a total of 4K 32-bit words $\Rightarrow$ cache size is $4 \times k \times 32/8$ (we divided by 8 to convert bits to byte, and since we have k which is equal to 2^{10} , we will convert all numbers to base two) $= 2^2 \times 2^{10} \times 2^2 = 2^{14}$ bytes

So the total number of lines is $2^{14}/4 = 2^{12}$ lines totally (we divided by 4 because the line size is 4 bytes).

So, the total number of sets is $2^{12}/4 = 2^{10}$ sets (we divided by 4 because each set is made up of 4 lines (4 way set associative))

$\Rightarrow$ 10 bits are needed to identify a set

We can now deduce the Tag length which is $24 - 2 - 10 = 12$ (we subtracted by 2 because we need two bits to identify a byte in a set, and we subtracted by 10 because we need 10 bits to identify a set in the cache).

2. Assume a direct mapped Cache with 16, one word blocks that is initially empty. A series of word address 1, 4, 8, 5, 20, 17, 19, 56, 9, 11, 4, 43, 5, 6, 9 and 17 are issued. Assuming the cache is initially empty, label each reference in the list as a hit or miss and show the final contents of the cache

Solution:

$1 \bmod 16 = 1$	miss	block 1 = contents of address 1
$4 \bmod 16 = 4$	miss	block 4 = contents of address 4
$8 \bmod 16 = 8$	miss	block 8 = contents of address 8
$5 \bmod 16 = 5$	miss	block 5 = contents of address 5
$20 \bmod 16 = 4$	miss	block 4 = replaced with contents of address 20
$17 \bmod 16 = 1$	miss	block 1 = replaced with contents of address 17
$19 \bmod 16 = 3$	miss	block 3 = contents of address 19
$56 \bmod 16 = 8$	miss	block 8 = replaced with contents of address 56
$9 \bmod 16 = 9$	miss	block 9 = contents of address 9
$11 \bmod 16 = 11$	miss	block 11 = contents of address 11
$4 \bmod 16 = 4$	miss	block 4 = replaced with contents of address 4
$43 \bmod 16 = 11$	miss	block 11 = replaced with contents of address 43
$5 \bmod 16 = 5$	hit	
$6 \bmod 16 = 6$	miss	block 6 = contents of address 6
$9 \bmod 16 = 9$	hit	
$\bmod 16 = 1$	hit	

3. Consider a two-level memory hierarchy, M1 and M2, with access times t_1 and t_2 costs per byte C_1 and C_2 , and capacities S_1 and S_2 , respectively. The cache hit ratio $h_1 = 0.95$ at the first level, and $h_2 = 1$.

Suppose $t_1 = 20$ ns, t_2 is unknown, $S_1 = 512$ Kbytes, S_2 is unknown, $C_1 = \$0.01/\text{byte}$, and $C_2 = \$0.0005/\text{byte}$. The total cost of the cache and main memory is upper-bounded by \$12,000. The effective access time $t_{\text{eff}} = 40$ ns of this memory system.

- a) How large a capacity of M2 ($S_2 = ?$)?
- b) How fast a main memory ($t_2 = ?$)?

Solution

- a) $S_2 = ??$

$$C_{\text{total}} = C_1 S_1 + C_2 S_2$$

$$12000 = 0.01 * 512 * 10^3 + 0.0005 * S_2$$

$$S_2 = (12000 - 5120) / 0.0005 = 13760000 \text{ bytes}$$

$$S_2 = 13.76 \text{ Mbytes}$$

- b) $t_2 = ??$

$$t_{\text{eff}} = h_1 t_1 + (1 - h_1) h_2 t_2$$

$$40 = 0.95 * 20 + 0.05 * t_2$$

$$t_2 = (40 - 0.95 * 20) / 0.05 = 420 \text{ ns}$$

4. Given a 4-way 32KB cache with 128 sets. For a 48-bit architecture, how big the entire tag array (in bits or bytes) is needed?

Solution

We need to calculate the entire tag array which is the total number of bits used for tags in all the cache which is equal to (number of lines * size of tag in bits). Note that each line has its own tag. First, we need to know the tag size which is equal to (48 - number of bits needed to identify a byte in a line - number of bits needed to identify a set)

4 way means that we have 4 lines in each set.

Number of sets = 128 = 2^7 . So, we need 7 bits to identify a set.

Now we need to know how many bits we need to identify a byte in a line, but we don't know the size of a line, so we will find first the size of a line in bytes.

Number of sets = 128. So, total number of lines = $128 * 4 = 512$

Line size = 32KB/512. Since $k = 2^{10}$, so, we need to convert all the numbers to base 2. Note that $512 = 2^7 * 2^2 = 2^9$

So, Line size = $32KB/512 = 2^5 * 2^{10} / 2^9 B = 2^{15}/2^9 B = 2^6 B \Rightarrow$ So, we need 6 bits to identify a byte in a line.

Each tag has = $48 - 7 - 6 = 35$ bits

Total tag array for 4 ways = $35 * 512 = 17920$ bits = 2240 Bytes = 2.1875 Kbytes (to convert from bytes to KB, we divide by 1024)

5. Following the previous question (4), now the architects decide to alter the cache into a **fully-associative** cache but keeping the 32KB capacity and each cache line size the same. For the same 48-bit architecture, how big the **entire tag array (in bits or bytes)** is needed?

Solution

For a fully associative cache, there is one single set. So, we don't need any bit to identify a set.

Tag size = 48 - number of bits needed to identify a byte in a line - number of bits needed to identify a set

Tag size = 48 - 6 - 0 = 42 bits

Total tag array for such a cache = 42 * 512 = 21504 bits = 2688 bytes = 2.625 KB

Question04:

5.10 For the Hamming code shown in Figure 5.10, show what happens when a check bit rather than a data bit is in error?

Answer:

The stored word is 001101001111, as shown in Figure 5.10. Now suppose that the only error is in C8, so that the fetched word is 001111001111. Then the received block results in the following table:

Position	12	11	10	9	8	7	6	5	4	3	2	1
Bits	D8	D7	D6	D5	C8	D4	D3	D2	C4	D1	C2	C1
Block	0	0	1	1	1	1	0	0	1	1	1	1
Codes			1010	1001		0111				0011		

The check bit calculation after reception:

Position	Code
Hamming	1111
10	1010
9	1001
7	0111
3	0011
XOR = syndrome	1000

The nonzero result detects an error and indicates that the error is in bit position 8, which is check bit C8.

5.11 Suppose an 8-bit data word stored in memory is 11000010. Using the Hamming algorithm, determine what check bits would be stored in memory with the data word. Show how you got your answer.

Answer:

Data bits with value 1 are in bit positions 12, 11, 5, 4, 2, and 1:

Position	12	11	10	9	8	7	6	5	4	3	2	1
Bits	D8	D7	D6	D5	C8	D4	D3	D2	C4	D1	C2	C1
Block	1	1	0	0		0	0	1		0		
Codes	1100	1011						0101				

The check bits are in bit numbers 8, 4, 2, and 1.

Check bit 8 calculated by values in bit numbers: 12, 11, 10 and 9

Check bit 4 calculated by values in bit numbers: 12, 7, 6, and 5

Check bit 2 calculated by values in bit numbers: 11, 10, 7, 6 and 3

Check bit 1 calculated by values in bit numbers: 11, 9, 7, 5 and 3

Thus, the check bits are: 0 0 1 0

5.12 For the 8-bit word 00111001, the check bits stored with it would be 0111. Suppose when the word is read from memory, the check bits are calculated to be 1101. What is the data word that was read from memory?

00011001 is the data word that was read from memory.

- 8-bit word = 00111001, Check bits stored = 0111.
- There are four bits and it's position will be
- $2^0 = 1 \Rightarrow p_1 = 1$
- $2^1 = 2 \Rightarrow p_2 = 1$
- $2^2 = 4 \Rightarrow p_4 = 1$
- $2^3 = 8 \Rightarrow p_8 = 0$.

- Encoded string will be:

$$d_{12} = 0; d_{11} = 0; d_{10} = 1; d_9 = 1;$$

$$d_8 = p_8 = 1; d_7 = 1; d_6 = 0; d_5 = 0;$$

$$d_4 = p_4 = 1; d_3 = 1; d_2 = p_2 = 1; d_1 = p_1 = 1.$$

- $\Rightarrow d_{12} = 0; d_{11} = 0; d_{10} = 1; d_9 = 1;$

$$d_8 = 1; d_7 = 1; d_6 = 0; d_5 = 0;$$

$$d_4 = 1; d_3 = 1; d_2 = 1; d_1 = 1.$$

- Check bits at other end = 1101.

- $XOR(0111, 1101) = 1010$. 10th bit will change.

- So, new encoded string will be:

$$d_{12} = 0; d_{11} = 0; d_{10} = 0; d_9 = 1;$$

$$d_8 = 1; d_7 = 1; d_6 = 0; d_5 = 0;$$

$$d_4 = 1; d_3 = 1; d_2 = 1; d_1 = 1.$$

- Hence the data word that was read from memory will be **00011001**.

Answer:

The Hamming Word initially calculated was:

bit number:

12	11	10	9	8	7	6	5	4	3	2	1
0	0	1	1	0	1	0	0	1	1	1	1

Doing an exclusive-OR of 0111 and 1101 yields 1010 indicating an error in bit 10 of the Hamming Word. Thus, the data word read from memory was 00011001.

5.13 How many check bits are needed if the Hamming error correction code is used to detect single bit errors in a 1024-bit data word?

Answer:

Need K check bits such that $2^k - 1 \geq 1024 + K$.

The minimum value of K that satisfies this condition is 11.

Question 05:

7.7 A microprocessor scans the status of an output I/O device every 20 ms. This is accomplished by means of a timer alerting the processor every 20 ms. The interface of the device includes two ports: one for status and one for data output. How long does it take to scan and service the device given a clocking rate of 8 MHz? Assume for simplicity that all pertinent instruction cycles take 12 clock cycles.

ANSWER:

Time to scan the device = Number of instructions x clock cycles per instruction

Frequency of processor = 8 MHz.

Clock cycle = 1 / frequency.

Time for one clock cycle = $1 / 8 \times 10^6 = 0.125 \times 10^{-6} = 0.125 \mu\text{s}$

The number of clock cycles of an instruction cycle takes 12. The time taken for 12 clock cycles is $12 \times 0.125 = 1.5 \mu\text{s}$.

Time take for clock cycles per instruction = $1.5 \mu\text{s}$.

In order to check the status of the device only one input instruction is required. Then there are two other instructions that will take place which will be when the device is examining the register contents and an output instruction when the device is ready to output information. Total number of instructions = 3.

Substitute 3 in the number of instructions and $1.5 \mu\text{s}$ for clock cycles.

Time taken = $3 \times 1.5 \mu\text{s} = 4.5 \mu\text{s}$.

Time taken to scan and service device would be $4.5 \mu\text{s}$.

7.10 Consider a system employing interrupt-driven I/O for a particular device that transfers data at an average of 8 KB/s on a continuous basis.

- a. Assume that interrupt processing takes about 100 s (i.e., the time to jump to the interrupt service routine (ISR), execute it, and return to the main program). Determine what fraction of processor time is consumed by this I/O device if it interrupts for every byte.
- b. Now assume that the device has two 16-byte buffers and interrupts the processor when one of the buffers is full. Naturally, interrupt processing takes longer, because the ISR must transfer 16 bytes. While executing the ISR, the processor takes about 8 s for the transfer of each byte. Determine what fraction of processor time is consumed by this I/O device in this case.
- c. Now assume that the processor is equipped with a block transfer I/O instruction such as that found on the Z8000. This permits the associated ISR to transfer each byte of a block in only 2 s. Determine what fraction of processor time is consumed by this I/O device in this case.

- 7.10 a. The device generates 8000 interrupts per second or a rate of one every 125 μ s. If each interrupt consumes 100 μ s, then the fraction of processor time consumed is $100/125 = 0.8$
- b. In this case, the time interval between interrupts is $16 \times 125 = 2000 \mu$ s. Each interrupt now requires 100 μ s for the first character plus the time for transferring each remaining character, which adds up to $8 \times 15 = 120 \mu$ s, for a total of 220 μ s. The fraction of processor time consumed is $220/2000 = 0.11$
- c. The time per byte has been reduced by 6 μ s, so the total time reduction is $16 \times 6 = 96 \mu$ s. The fraction of processor time consumed is therefore $(220 - 96)/2000 = 0.062$. This is an improvement of almost a factor of 2 over the result from part (b).

7.12 A DMA module is transferring characters to memory using cycle stealing, from a device transmitting at 9600 bps. The processor is fetching instructions at the rate of 1 million

instructions per second (1 MIPS). By how much will the processor be slowed down due to the DMA activity?

The DMA combines (assembles) one word from one consecutive character (byte) so we get

9600 chars/s = 9600 bytes/s = 1200 words/s = 1200 W/s (assuming 1word=1 Byte)

If we assume that one CPU instruction is one word wide then
1 million instructions/s = 1 million words/s = 10^6 W/s

So we have 1200 words received during one second and ($10^6 - 1200$) words processed by the CPU (while DMA is transferring a word, the CPU cannot fetch the instruction so we have to subtract the number of words transferred by DMA) and we get
 $(1200 \text{ w/s}) / (10^6 \text{ w/s} - 1200 \text{ w/s}) = 1200 / (10^6 - 1200) = 1200 / 998800 = 0.0012014417...$

So Convert 0.0012 this in ms. it will come 1.2 ms

7.13 Consider a system in which bus cycles takes 500 ns. Transfer of bus control in either direction, from processor to I/O device or vice versa, takes 250 ns. One of the I/O devices has a data transfer rate of 50 KB/s and employs DMA. Data are transferred one byte at a time.

- a. Suppose we employ DMA in a burst mode. That is, the DMA interface gains bus mastership prior to the start of a block transfer and maintains control of the bus until the whole block is transferred. For how long would the device tie up the bus when transferring a block of 128 bytes?
- b. Repeat the calculation for cycle-stealing mode

- 7.13 a. For the actual transfer, the time needed is $(128 \text{ bytes}) / (50 \text{ KBps}) = 2.56 \text{ ms}$. Added to this is the time to transfer bus control at the beginning and end of the transfer, which is $250 + 250 = 500 \text{ ns}$. This additional time is negligible, so that the transfer time can be considered as 2.56 ms.
- b. The time to transfer one byte in cycle stealing mode is $250 + 500 + 250 = 1000 \text{ ns} = 1 \mu\text{s}$. Total amount of time the bus is occupied for the transfer is $128 \mu\text{s}$. This is less than the result from part (a) by a factor of 20.

Question 06:

8.4 Consider a fixed partitioning scheme with equal-size partitions of 2^{16} bytes and a total main memory size of 2^{24} bytes. A process table is maintained that includes a pointer to a partition for each resident process. How many bits are required for the pointer?

Solution:

The number of partitions equals the number of bytes of main memory divided by the number of bytes in each partition: $2^{24}/2^{16} = 2^8$. Eight bits are needed to identify one of the 2^8 partitions.

8.6 Suppose the page table for the process currently executing on the processor looks like the following. All numbers are decimal, everything is numbered starting from zero, and all addresses are memory byte addresses. The page size is 1024 bytes.

Virtual page number	Valid bit	Reference bit	Modify bit	Page frame number
0	1	1	0	4
1	1	1	1	7
2	0	0	0	—
3	1	0	0	2
4	0	0	0	—
5	1	0	1	0

- Describe exactly how, in general, a virtual address generated by the CPU is translated into a physical main memory address.
- What physical address, if any, would each of the following virtual addresses correspond to? (Do not try to handle any page faults, if any.)
 - 1052
 - 2221
 - 5499

Solution:

- Split binary address into virtual page number and offset; use VPN as index into page table; extract page frame number; concatenate offset to get physical memory address

- b. (i) $1052 = 1024 + 28$ maps to VPN 1 in PFN 7, ($7 \times 1024 + 28 = 7196$)
(ii) $2221 = 2 \times 1024 + 173$ maps to VPN 2, page fault
(iii) $5499 = 5 \times 1024 + 379$ maps to VPN 5 in PFN 0, ($0 \times 1024 + 379 = 379$)

8.7 Give reasons that the page size in a virtual memory system should be neither very small nor very large.

Solution:

With very small page size, there are two problems: (1) Because very little data is brought in with each page, there will need to be a lot of I/O to bring in the many small pages. (2) The overhead (page table size, length of field for page number) will be disproportionately high.

If pages are very large, main memory will be wasted because the principle of locality suggests that only a small part of the large page will be used.

8.14 A computer has a cache, main memory, and a disk used for virtual memory. If a referenced word is in the cache, 20 ns are required to access it. If it is in main memory but not in the cache, 60 ns are needed to load it into the cache, and then the reference is started again. If the word is not in main memory, 12 ms are required to fetch the word from disk, followed by 60 ns to copy it to the cache, and then the reference is started again. The cache hit ratio is 0.9 and the main-memory hit ratio is 0.6. What is the average time in ns required to access a referenced word on this system?

Solution:

8.14 There are three cases to consider:

Location of referenced word	Probability	Total time for access in ns
In cache	0.9	20
Not in cache, but in main memory	$(0.1)(0.6) = 0.06$	$60 + 20 = 80$
Not in cache or main memory	$(0.1)(0.4) = 0.04$	$12\text{ms} + 60 + 20 = 12000080$

So the average access time would be:

$$\text{Avg} = (0.9)(20) + (0.06)(80) + (0.04)(12000080) = 480026 \text{ ns}$$

Consider a logical address space of 32 pages of 2048 words each, mapped into a physical memory of 512 frames.

1. How many bits are there in logical address?
2. How many bits are there in physical address?

Solution:

1.

$m = ???$

Size of logical address space = $2^m = \# \text{ of pages} \times \text{page size}$

$$2^m = 32 \times 2048$$

$$2^m = 2^5 \times 2^{11} = 2^{16} \Rightarrow m = 16 \text{ bit}$$

2.

Let x is number of bits in the physical address

$x = ???$

Size of physical address space = 2^x

Size of physical address space = $\# \text{ of frames} \times \text{frame size}$; (frame size = page size)

$$\text{Size of physical address space} = 512 \times 2048$$

$$2^x = 2^9 \times 2^{11} = 2^{20} \Rightarrow \text{number of required bits in the physical address} = x = 20 \text{ bit}$$

8.17 Consider a paged logical address space (composed of 32 pages of 2 KBytes each) mapped into a 1-MByte physical memory space.

- a. What is the format of the processor's logical address?
- b. What is the length and width of the page table (disregarding the "access rights" bits)?
- c. What is the effect on the page table if the physical memory space is reduced by half?

Solution:

8.17 a.

page number (5)	offset (11)
--------------------	-------------

- b. 32 entries, each entry is 9 bits wide.
- c. If total number of entries stays at 32 and the page size does not change, then each entry becomes 8 bits wide.

Question 07:

- 13.1 Given the following memory values and a one-address machine with an accumulator, what values do the following instructions load into the accumulator?
- Word 20 contains 40.
 - Word 30 contains 50.
 - Word 40 contains 60.
 - Word 50 contains 70.
- a. LOAD IMMEDIATE 20
 - b. LOAD DIRECT 20
 - c. LOAD INDIRECT 20
 - d. LOAD IMMEDIATE 30
 - e. LOAD DIRECT 30
 - f. LOAD INDIRECT 30

Solution:

a. 20 b. 40 c. 60 d. 30 e. 50 f. 70

- 13.3 An address field in an instruction contains decimal value 14. Where is the corresponding operand located for
- a. immediate addressing?
 - b. direct addressing?
 - c. indirect addressing?
 - d. register addressing?
 - e. register indirect addressing?

Solution:

- a. the address field
- b. memory location 14
- c. the memory location whose address is in memory location 14
- d. register 14
- e. the memory location whose address is in register 14

- 13.4** Consider a 16-bit processor in which the following appears in main memory, starting at location 200:

200	Load to AC	Mode
201	500	
202	Next instruction	

The first part of the first word indicates that this instruction loads a value into an accumulator. The Mode field specifies an addressing mode and, if appropriate, indicates a source register; assume that when used, the source register is R1, which has a value of 400. There is also a base register that contains the value 100. The value of 500 in location 201 may be part of the address calculation. Assume that location 399 contains the value 999, location 400 contains the value 1000, and so on. Determine the effective address and the operand to be loaded for the following address modes:

- a. Direct
- b. Immediate
- c. Indirect
- d. Register
- e. Register direct
- f. Register indirect
- g. Register indirect

Solution:

	EA	Operand		EA	Operand	
a.	500	1100	f.	R1	400	
b.	201	500		g.	400	1000
c.	1100	1700				

- 13.5** A PC-relative mode branch instruction is 3 bytes long. The address of the instruction, in decimal, is 256028. Determine the branch target address if the signed displacement in the instruction is -31.

Solution:

Recall that relative addressing uses the contents of the program counter, which points to the next instruction after the current instruction. In this case, the current instruction is at decimal address 256028 and is 3 bytes long, so the PC contains 256031. With the displacement of -31, the effective address is 256000.

- 13.13** Assume a stack-oriented processor that includes the stack operations PUSH and POP. Arithmetic operations automatically involve the top one or two stack elements. Begin with an empty stack. What stack elements remain after the following instructions are executed?

PUSH	4
PUSH	7
PUSH	8
ADD	
PUSH	10
SUB	
MUL	

Solution:

Instruction	Stack (top on left)
PUSH 4	4
PUSH 7	7, 4
PUSH 8	8, 7, 4
ADD	15, 4
PUSH 10	10, 15, 4
SUB	5, 4
MUL	20

Question 08:

APPLICATIONS (1 OF 4)

- Task: Convert the character in AL to upper case.
- Solution: Use the AND instruction to clear bit 5.

```
mov al, 'a'          ; AL = 01100001b
and al, 11011111b    ; AL = 01000001b
```

APPLICATIONS (2 OF 4)

- Task: Convert a binary decimal byte into its equivalent ASCII decimal digit.
- Solution: Use the OR instruction to set bits 4 and 5.

```
mov al, 6             ; AL = 00000110b
or  al, 00110000b     ; AL = 00110110b
```

The ASCII digit '6' = 00110110b

APPLICATIONS (3 OF 4)

- Task: Jump to a label if an integer is even.
- Solution: AND the lowest bit with a 1. If the result is Zero, the number was even.

```
mov ax, wordVal
and ax, 1             ; low bit set?
jz  EvenValue         ; jump if Zero flag set
```

APPLICATIONS (4 OF 4)

- Task: Jump to a label if the value in AL is not zero.
- Solution: OR the byte with itself, then use the JNZ (jump if not zero) instruction.

```
or  al, al
jnz IsNotZero         ; jump if not zero
```

ORing any number with itself does not change its value.

Question 09:

EXAMPLE

- Implement the following pseudocode in assembly language. All values are unsigned:

```
if( ebx <= ecx )
{
    eax = 5;
    edx = 6;
}
```

```
cmp ebx, ecx
ja  next      ; jump if above (if leftOp > rightOp)
mov eax, 5
mov edx, 6
next:
```

EXAMPLE

- Implement the following pseudocode in assembly language. All values are 32-bit signed integers:

```
if( var1 <= var2 )
    var3 = 10;
else
{
    var3 = 6;
    var4 = 7;
}
```

```
mov eax,var1
cmp eax,var2
jle L1      ; Jump if less than or equal
            ; leftOp <= rightOp
mov var3,6
mov var4,7
jmp L2
L1: mov var3,10
L2:
```

Question 10:

EXERCISE

```
while( ebx <= val1)
{
    ebx = ebx + 5;
    val1 = val1 - 1
}
```

- Implement the following loop, using unsigned 32-bit integers:

```
_while:
    cmp ebx,val1    ; check loop condition
    ja _endwhile   ; false? exit loop
    add ebx,5       ; body of loop
    dec val1
    jmp while       ; repeat the loop
_endwhile:
```